PRODUCTION RAG GUIDE

Most Asked Questions on LangChain for RAG

10 essential production interview questions, LCEL internals, multi-hop retrieval, and framework tradeoffs.

10 Q&A

Architecture Flows

LCEL & LangGraph

10 Questions Covered in This Guide

From core abstractions to LCEL internals and production limits:

PART 1: ARCHITECTURE

- Q1. Core RAG Components
- Q2. Abstraction Layers
- Q3. Production Limitations
- Q4. Multi-Step Reasoning
- Q5. Memory in RAG

PART 2: ADVANCED

- Q6. Streaming Support
- Q7. Custom Code vs LC
- Q8. Hybrid Search & Rerank
- Q9. LCEL Internals & Pipe
- Q10. LangGraph State Loops

✅ **Detailed answers & architecture workflows inside!**

🎓 Check caption to get interview kit with full answers and more questions

QUESTION 01

Core Components & Interactions

> LangChain wires together the building blocks of a RAG system so you do not have to write custom plumbing code.

ESSENTIAL PIPELINE BLOCKS

- **Document Loaders:** Ingest PDFs, S3, Notion, URLs into `Document` objects.
- **Text Splitters:** Break text into chunk-sized Documents (e.g. `RecursiveCharacterTextSplitter`).
- **Vector Stores & Embeddings:** Store embeddings and expose `similarity_search()` .
- **Retrievers & Chains:** Fetch top-k chunks and feed structured prompts to LLM.

WORKFLOW DESIGN: COMPONENT PIPELINE



Check caption to get interview kit with full answers and more questions

QUESTION 02

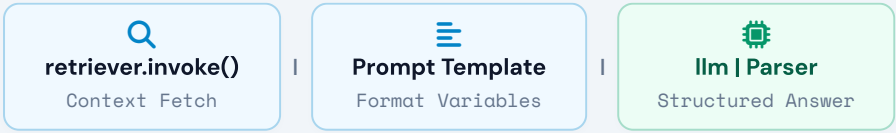
LangChain Abstraction Layers

LangChain provides standardized interfaces so developers can swap retrieval backends, prompt templates, and LLM providers without rewriting business logic.

MODERN RUNNABLE ABSTRACTIONS

- **Retrievers:** Implement `Runnable` interface via `.invoke(query)` across Pinecone, FAISS, or Qdrant.
- **Prompts:** `ChatPromptTemplate` dynamically injects `{context}` and `{question}` variables.
- **Generation:** Unified `llm.invoke()` works across OpenAI, Anthropic, and local models.
- **Output Parsers:** Seamlessly format raw outputs into structured JSON or Pydantic.

</> WORKFLOW DESIGN: RUNNABLE COMPOSITION



Check caption to get interview kit with full answers and more questions

QUESTION 03

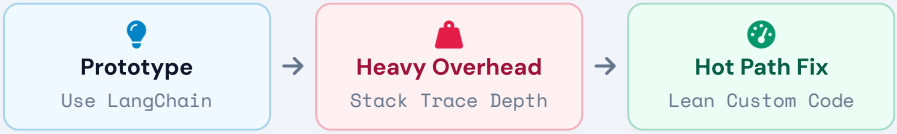
Production Scale Limitations

> LangChain is great for prototyping. But in large-scale, high-concurrency production systems, developers hit specific real-world friction.

KEY PRODUCTION FRICTION POINTS

- ✕ **Hidden Overhead & Latency:** Wrapper objects add latency sensitive for under-100ms systems.
- ✕ **Rapid API Shifts:** Breaking changes across v0.0.x, v0.1.x, and v0.2.x complicate long-term maintenance.
- ✕ **Debugging Depth:** Stack traces through internal chains are deep and hard to interpret.
- ✕ **Dependency Bloat:** Installs dozens of packages even for minimal feature usage.

WORKFLOW DESIGN: PROTOTYPE VS HOT-PATH CODE



Check caption to get interview kit with full answers and more questions

QUESTION 04

Multi-Step & Multi-Hop RAG

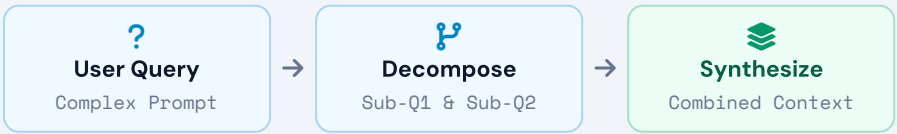
> Multi-step reasoning decomposes complex user questions into sub-queries, retrieves context in multiple hops, and synthesizes final answers.

3-STEP EXECUTION PATTERN

- **Step 1: Question Decomposition:** LLM splits complex prompt into 2 to 3 focused sub-questions.
- **Step 2: Multi-Hop Retrieval:** Execute vector retrieval for each sub-question & deduplicate chunks.
- **Step 3: Synthesis:** Combine retrieved evidence and generate cohesive final output.

✓ **Note:** Modern systems use LCEL and LangGraph state machines instead of subclassing `Chain` .

WORKFLOW DESIGN: MULTI-HOP PIPELINE



Check caption to get interview kit with full answers and more questions

QUESTION 05

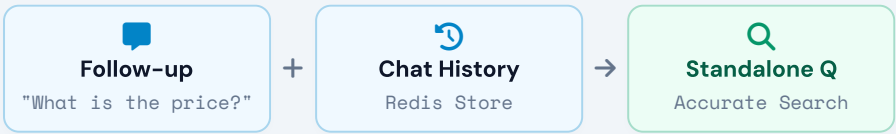
Conversational Memory in RAG

> Memory allows conversational RAG to maintain context across turns. The system reformulates user follow-up questions before vector search.

MEMORY TYPES & RAG INTEGRATION

- **Buffer Memory:** Keeps full history. Simple, but quickly overflows context windows.
- **Window & Summary:** Keeps last K turns or uses LLM to summarize older history.
- **Query Reformulation:** Turn "What is the price?" into "What is the price of Pro plan?" before retrieval.
- **Production Practice:** Persist history in Redis / PostgreSQL, not in-memory Python objects.

WORKFLOW DESIGN: REFORMULATED RETRIEVAL



Check caption to get interview kit with full answers and more questions

QUESTION 06

Implementing Streaming Responses

> Streaming sends tokens to the user as generated, dramatically lowering time-to-first-token (TTFT) perceived latency.

LCEL & FASTAPI SERVER SENT EVENTS (SSE)

Async Streaming: Use `chain.astream()` inside FastAPI `StreamingResponse` with `text/event-stream`.

Retrieval Non-Streaming: Vector retrieval occurs before generation. User waits for retrieval, then tokens stream.

UX Best Practice: Send "Searching context..." event during retrieval, then stream LLM chunks.

WORKFLOW DESIGN: ASYNC TOKEN STREAM



Check caption to get interview kit with full answers and more questions

QUESTION 07

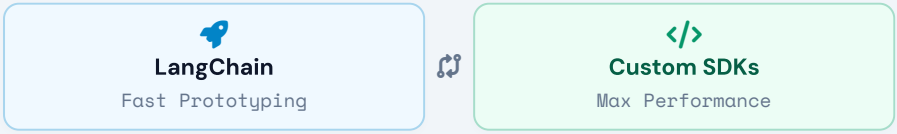
When to Build a Custom Pipeline

> Choose a custom pipeline with direct SDK calls over LangChain when performance, stability, and exact token control are top priorities.

DECISION CRITERIA

- **Strict Latency Requirements:** Under-100ms targets cannot afford framework wrapper overhead.
- **Complex Retrieval:** Custom multi-stage scoring or ranking that breaks framework interfaces.
- **API Stability:** High-volume production where breaking dependency updates are unacceptable.
- **Token Cost Optimization:** Need absolute control over prompt tokens sent to provider APIs.

WORKFLOW DESIGN: FRAMEWORK TRADEOFF



Check caption to get interview kit with full answers and more questions

QUESTION 08

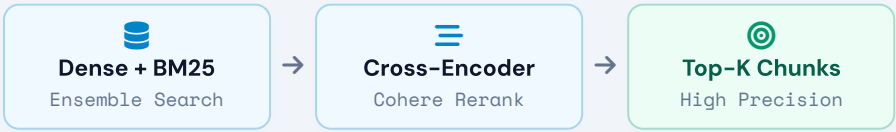
Hybrid Search & Reranking in LangChain

Dense embeddings capture semantic meaning, while sparse search (BM25) captures exact keywords. Combining both with reranking yields maximum retrieval accuracy.

HYBRID IMPLEMENTATION ARCHITECTURE

- **EnsembleRetriever:** Combines `BM25Retriever` and `VectorStoreRetriever` with weights (e.g. 0.5 / 0.5).
- **ContextualCompressionRetriever:** Wraps ensemble retriever with a cross-encoder model.
- **Reranker:** Uses Cohere Rerank or HuggingFace CrossEncoder to re-score top candidate chunks.

WORKFLOW DESIGN: HYBRID + RERANK PIPELINE



Check caption to get interview kit with full answers and more questions

QUESTION 09

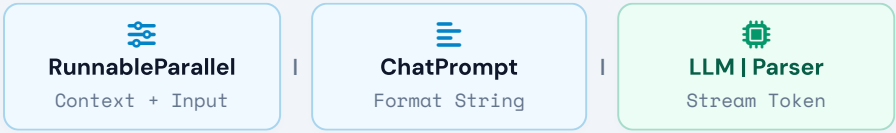
LCEL & Runnable Internals

> LangChain Expression Language (LCEL) uses the pipe operator | to chain components implementing the `Runnable` protocol.

</> WHY LCEL REPLACED LEGACY CHAINS

- **Unified Interface:** Every component implements `invoke`, `stream`, `batch`, and `ainvoke`.
- **Auto Parallelization:** `RunnableParallel` executes independent branches automatically.
- **First-class Streaming & Tracing:** Native support for streaming without manual callbacks.

WORKFLOW DESIGN: PIPE OPERATOR COMPOSITION



Check caption to get interview kit with full answers and more questions

QUESTION 10

Migrating to LangGraph State Loops

> Classic LangChain DAGs are strictly acyclic. Agentic RAG requiring loops (self-correction, document grading, query rewrite) is modeled in LangGraph.

CORE LANGGRAPH CONCEPTS

- **State Graph:** Central state schema updated by each node step in the graph.
- **Cyclic Edges:** Route conditionally back to retrieval or query rewriting if grading fails.
- **State Checkpointing:** Built-in persistence for human-in-the-loop approvals and time travel.

WORKFLOW DESIGN: SELF-CORRECTION LOOP



Check caption to get interview kit with full answers and more questions

RAG Architecture Blueprint

Key takeaways for building production-grade LangChain systems:

</> USE LCEL

Replace legacy Chain subclasses with modern Pipe Runnables.

HYBRID SEARCH

Combine BM25 + Vector embeddings with Cohere Reranker.

LANGGRAPH

Use state machines for self-correction loops & multi-agent routing.

REDIS MEMORY

Store chat history in external DBs, not Python in-memory objects.

PracticAI Engineering Series

Building production-grade AI systems, RAG architectures, and agentic workflows.

Save • Repost • + Follow PracticAI

Check caption to get interview kit with full answers and more questions